Home

Antigravity 2.0

Antigravity CLI

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

Home

Welcome to Google Antigravity

Choose Your Surface

Google Antigravity offers multiple product surfaces tailored to your specific development workflow. Select the interface that best fits your needs:

Antigravity 2.0

Your standalone desktop command center for your agents. Start agents inside Projects, work across multiple workspaces and worktrees, and orchestrate complex tasks using parallel local subagents.

- Key Features: Asynchronous task management, Scheduled Tasks (Cron sidecars), and voice transcription.
- Get Started: Read the Getting Started Guide

Antigravity CLI

The lightweight, keyboard-centric Terminal User Interface surface. It brings the same core agentic capabilities as the desktop app directly to your terminal workflow, making it perfect for fast interactions and SSH sessions.

- Key Features: High-speed prompt shortcuts, custom keybindings, and parallel subagent management.
- Get Started: Explore the CLI Quick Overview

Antigravity SDK

A programmatic Python framework for researchers and developers who want complete control over their agent deployments. Custom build an agent, register custom tools, and implement lifecycle hooks, all on top of the Antigravity Harness.

- Key Features: Declarative safety policies, inspect/decide/transform hooks, and programmatic subagent spawning.
- Get Started: Review the SDK Overview

Antigravity IDE

The fully-featured, AI-powered developer environment. Standardize your daily coding with powerful tightly integrated coding agents, deep context awareness, tools like MCP and skills, and more.

- Get Started: Read the IDE Getting Started Guide

Core Agent Capabilities

Every Antigravity surface runs on a shared, highly-optimized agent harness co-trained with Gemini models:

- Gemini 3.5 Flash: Powering all local agents with SOTA speed, reasoning, and context window capacity.
- Asynchronous Subagents: Allows the main agent to delegate parallel background tasks to concurrent subagents without blocking your flow.
- Visual Artifacts: Track and verify agent output (plans, code diffs, browser recordings) with high-fidelity visual reports, keeping you informed every step of the way.
- Security by Design: Secure local execution via safe defaults, local proxying, and granular tool approval gates.
- Google Integrations: We partner with product teams across Google to provide curated bundles of skills, MCP servers, and extensions that make building on Google platforms frictionless.
- Android: Editor extension, CLI integrations, and Android developer skills.
- Firebase: Curated skills for Firebase Firestore, Cloud Functions, and more.
- Web: Chrome and Web MCP servers for autonomous browser research.

- Science: Specialized DeepMind biology and chemistry skills to accelerate scientific workflows.
- AGY SDK: Skills that optimize your agent's ability to use the Antigravity SDK to build custom AI agents tailored to your workflow.

Overview

On this Page

Welcome to Google Antigravity

Choose Your Surface

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity 2.0
>Overview

Antigravity 2.0

Overview

As we go deeper into the agent-first era, Antigravity is evolving to further empower developers. Introducing Antigravity 2.0, a standalone desktop application tailored for managing AI agents that execute complex knowledge and coding tasks.

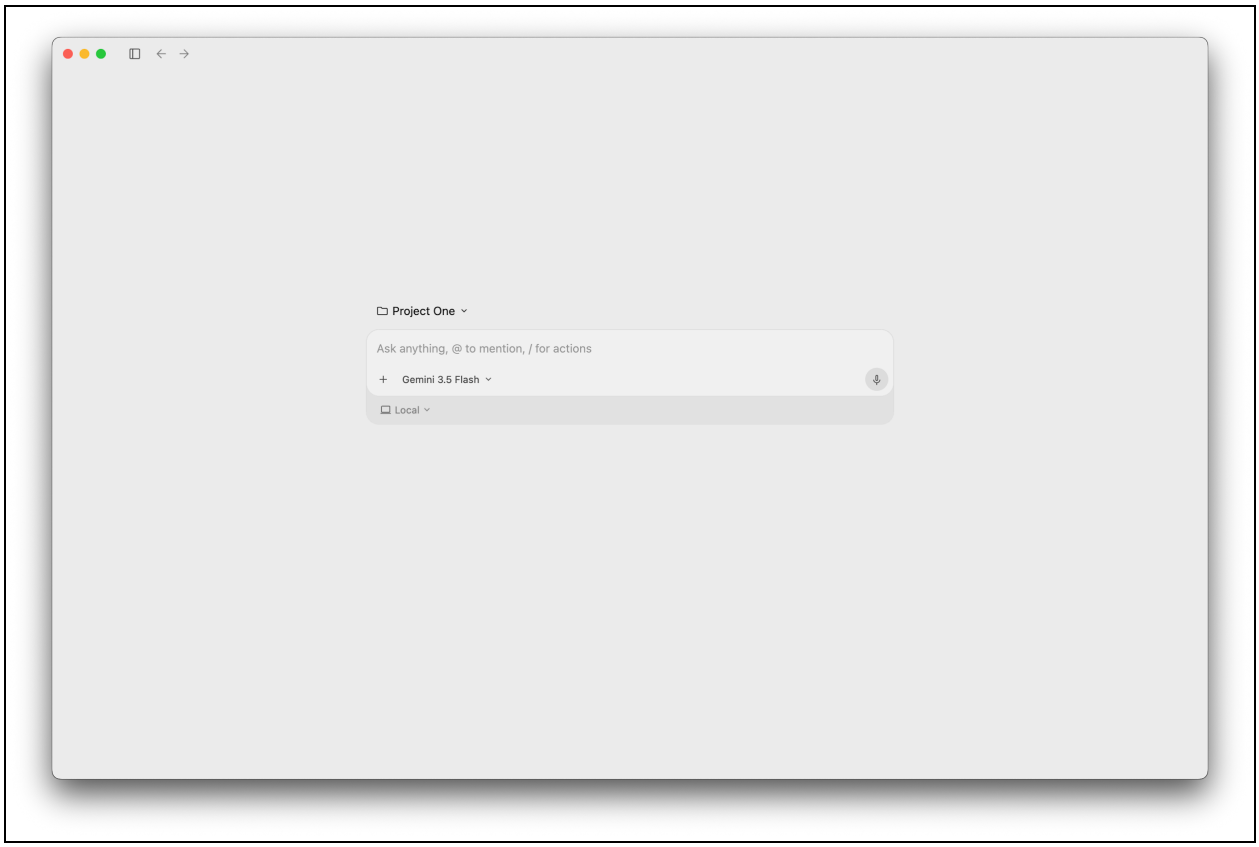
What is Antigravity 2.0?

Antigravity 2.0 serves as your AI agents' central command center, providing a unified platform to launch, monitor, and orchestrate their activities. Unlike its predecessor, the Agent Manager, Antigravity 2.0 is a standalone application that functions independently of an IDE.

Within this interface, you can orchestrate agents both synchronously and asynchronously to:

- Execute system commands
- Perform file read/write operations
- Conduct web searches
- Integrate with external tools via skills and MCP servers
- Manage subagents
- Interact with Chrome
- Create artifacts / implementation plans

Whether you are performing deep research or building new applications, Antigravity offers a streamlined interface for all types of knowledge work.



Getting Started

Ready to dive in? Check out the Getting Started guide to install Antigravity 2.0 and start your first project.

Getting Started

On this Page

Antigravity 2.0

Overview

Home

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity CLI Overview

The Antigravity CLI is the lightweight Terminal User Interface (TUI) surface of Antigravity. It brings the same core agentic capabilities as Antigravity 2.0 (such as multi-step reasoning, multi-file editing, tool calling, and conversation history) directly to your terminal.

Why Antigravity CLI?

Antigravity CLI brings the reasoning, execution, and orchestration capabilities of our shared agent harness directly into your local shell. While Antigravity 2.0 offers a comprehensive visual editor interface, the CLI is custom-built for speed, lightweight operation, and seamless integration with terminal-first workflows.

Platform comparison

Feature	Antigravity CLI	Antigravity 2.0

Primary interface	Keyboard-driven TUI	Visual desktop editor / IDE
Performance overhead	Near-zero, extremely lightweight	Standard desktop IDE footprint
Workflow focus	Fast local iterations, SSH, headless	Complete project management, visual workspace
Navigation	Universal keyboard shortcuts	Mouse and multi-panel layout
Remote usability	Native SSH, tmux, and terminal multiplexers	Local workspace or remote development containers

Integration features

Antigravity CLI operates in tandem with Antigravity 2.0, sharing configurations and enabling frictionless transitions between interfaces:

- Shared agent harness: Both environments run on the exact same agent core. Any enhancements to multi-step reasoning, tool usage, or code comprehension apply across both platforms.
- Shared settings sync: Your core preferences, permissions, and security configurations synchronize automatically across both interfaces. Updating a permission rule or standard configuration in one platform immediately updates the other.
- Conversation export: Seamlessly move active conversations between platforms. If a terminal session grows in complexity and requires visual orchestration, export the conversation to Antigravity 2.0 to continue with the visual editor interface.

Migrating from Gemini CLI

If you are transitioning from Gemini CLI, the onboarding process supports a one-time import to automatically migrate your existing Gemini CLI extensions, skills, and settings. To learn more, read Migrating from Gemini CLI.

Next steps

Explore the guides below to set up your environment and begin working with autonomous agents:

- Installation & Auth: Set up the CLI, configure enterprise parameters, and complete silent authentication.
- Getting Started: Explore the onboarding roadmap, first-launch setups, and core conceptual models.
- Tutorial: Run your first multi-file generation task with an active agent.
- Prompting & Interaction: Master multiline composing, prompt editing, and pasting terminal media.
- Reviewing Artifacts: Leverage transparency and review agent plans, diffs, and test runs.
- AI Credits: Configure and monitor AI Premium credits fallback, pricing links, and settings.
- Plugins & Skills: Create your own custom skills slash commands, manage hooks, and configure MCP servers.
- Best Practices: Master workflow pipelines, verification loops, and session course-corrections.

Screenshots

Getting Started

On this Page

Antigravity CLI Overview

Why Antigravity CLI?

Integration features

Migrating from Gemini CLI

Next steps

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity CLI

>Getting Started

Getting Started with Antigravity CLI

Welcome to Antigravity CLI! This guide provides a direct, high-level developer roadmap to install the client, launch the Terminal User Interface (TUI), and begin collaborating with autonomous agents.

Roadmap checklist

Complete the following sequential steps to launch your first session:

1. Install the client (fast path)

Run the appropriate fast-path command for your operating system:

macOS / Linux:

content_copy

```
curl -fsSL https://antigravity.google/cli/install.sh | bash
```

Windows (PowerShell):

content_copy

```
irm https://antigravity.google/cli/install.ps1 | iex
```

Windows (CMD):

content_copy

```
curl -fsSL https://antigravity.google/cli/install.cmd -o install.cmd && install.cmd && del install.cmd
```

By default, the installer registers the `agy` binary to your platform-specific directory:

- macOS / Linux: `~/.local/bin/agy`
- Windows: `C:\Users\<Username>\AppData\Local\agy\bin` (where `<Username>` represents your active Windows profile name).

info

Advanced Setup: For detailed enterprise credentials configuration, secure keyring auth permissions, proxy setups, or troubleshooting installation issues, consult the **Installation & Auth Guide**.

1. Launch the TUI inside a project

Open a fresh terminal window, navigate to your target project codebase directory, and execute the launcher command:

content_copy

```
agy
```

1. Complete the first-launch setup

On your very first launch, the TUI walks you through a brief interactive setup:

- Color Scheme: Select your preferred visual theme (Solarized, Dark, Solarized Light, or standard Terminal colors).
- Rendering Mode: Choose Alt-Screen mode (alternate buffer with full-screen scrolling) or Inline mode (sequential stream integrated with your terminal's history).
- Workspace Trust: Confirm that you trust the repository directory. Once confirmed, the agent indexes the files and stands ready.

1. Run your first agent task

Type the following instruction in the prompt box at the bottom of your TUI screen and press `Enter`:

content_copy

Write a simple python script to fetch web page text

The agent reads the workspace, reasons about the task, and proposes a plan. For a detailed step-by-step tutorial on reviewing code and running test commands inside the TUI, follow the Tutorial Guide.

Related resources

Optimize your local environment configurations and master advanced collaboration tools:

- Best Practices: Master verification loops, planning phases, rule files, and session checkpoints.
- Troubleshooting: Resolve common path, keyring, or SSH forwarding errors.
- CLI Reference: Dense reference sheets cataloging all slash commands, shortcuts, and JSON keys.

Installation & Auth

On this Page

Getting Started with Antigravity CLI

Roadmap checklist

Related resources

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Installation & auth

Install Antigravity CLI, configure enterprise requirements, and establish secure authenticated sessions.

Installation

Antigravity CLI runs natively on macOS, Linux, and Windows. Use the platform-specific scripts below to install or upgrade the binary on your system.

macOS and Linux

Execute the native installer script to download and install the executable to `~/.local/bin/agy`:

bash

content_copy

curl -fsSL https://antigravity.google/cli/install.sh | bash

Windows

The installation script registers the `agy` binary to your local user directory: `C:\Users\<Username>\AppData\Local\agy\bin` (where `<Username>` represents your active Windows user profile).

PowerShell: Open PowerShell and execute the following installation script:

```
powershell

content_copy

irm https://antigravity.google/cli/install.ps1 | iex
```

CMD: Open a standard Command Prompt and execute:

```
cmd

content_copy

curl -fsSL https //antigravity.google/cli/install.cmd -o install.cmd && install.cmd && del install.cmd
```

Installation flags

When executing the installation scripts, you can append the following customization flags:

- `--skip-aliases`: Bypasses shell profile alias purging (prevents the script from purging or updating legacy `agy` or `antigravity` shell aliases).
- `--skip-path`: Bypasses shell profile `PATH` appending (prevents the script from modifying your shell profile's dynamic environment variables).

Authentication workflows

Antigravity CLI uses secure credentials and token profiles to communicate with the shared agent harness.

Local silent keyring sign-in

When launching `agy` on your local machine, the CLI attempts to access your operating system's native secure keyring (such as Apple Keychain, Linux Secret Service/dbus, or Windows Credential Manager). If a valid token profile is found, the CLI authenticates your session silently without opening a browser.

If no saved session is found:

1. The CLI automatically launches your local default web browser.
2. Sign in using your approved account credentials.

Remote SSH OAuth flow

When running over SSH, the CLI detects the remote connection environment. Because it cannot launch a local web browser, the CLI initiates a manual URL loop:

1. Launch `agy` in your remote terminal session.
2. The CLI detects the SSH environment and prints a unique, secure authorization URL.
3. Copy this URL and paste it into a web browser on your local machine.
4. Sign in with your approved credentials and complete the authentication.
5. The browser displays a unique alphanumeric authorization code.
6. Copy this code, return to your remote SSH terminal, and paste it into the prompt.

Managing your session

Terminating your session clears active credentials and local cache directories.

Logging out

To disconnect your account and purge saved authentication profiles from your operating system's keyring, run the following command in the CLI prompt box:

```
text

content_copy

/logout
```

Next steps

Once you complete installation and authentication, start interacting with your local agent:

- Tutorial: Create and run a basic Python project with an agent.
- Prompting & Interaction: Explore multiline text editing, interrupt commands, and terminal media pasting.
- Permissions & Sandbox: Configure secure filesystem directories and command limits.

Tutorial

On this Page

- Installation & auth
- Installation
- Authentication workflows
- Managing your session
- Next steps

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

Antigravity CLI Tutorial

Learn how to launch Antigravity CLI, collaborate with an autonomous local agent, review generated files, and execute terminal test commands.

Overview

This guide walks you through a rapid onboarding exercise. You will direct an autonomous agent to create a Python utility script, review the changes, and verify its execution.

Step-by-step

1. Create a clean project directory and launch the Antigravity TUI

content_copy

```
mkdir agy-demo && cd agy-demo
```

```
agy
```

info

First Launch: If running `agy` for the first time, follow the terminal instructions to complete silent authentication. See Installation & Auth for troubleshooting details.

1. Prompt the agent to generate a Python scraping script

Type the following instruction in the prompt box at the bottom of your screen and press `Enter`:

content_copy

```
Write a simple python script to fetch web page text
```

The agent reads the workspace, determines that no files exist, and formulates a plan to create a script. You will see real-time updates as the agent performs reasoning and schedules actions.

1. Open the artifact review screen to inspect the proposed code

Once the agent finishes generating the file, a notification appears. Press `ctrl+r` to enter the Artifact Review screen.

- Navigate to the newly created `main.py` using `↑`/`↓`.
- Review the complete file content and diff.
- Press `y` to approve the creation of `main.py`.
- Press `Esc` to close the review panel and return to the primary prompt.

1. Execute a test command with the agent to verify the output

Direct the agent to run the Python script to verify its behavior. Type the following command in the prompt box and press `Enter`:

content_copy

```
Run the python script and show me the output
```

The agent proposes to run `python3 main.py`. Press `y` to confirm and execute the command. The agent runs the script locally and streams the standard output directly into your terminal screen.

1. Exit the Antigravity session

Once you complete your task, press `ctrl+d` (or type `/exit`) in the prompt box to close the TUI and restore your original shell session.

Next steps

Now that you have executed your first agent-assisted workflow, learn how to configure the CLI and master core concepts:

- Installation & Auth: Detailed instructions on installing `agy` and setting up SSH profiles.
- Prompting & Interaction: Best practices for multiline inputs, pasting media files, and active interrupt controls.
- Reviewing Artifacts: Deep dive into the "Trust through Transparency" architectural pattern.

Installation & Auth

Using AGY CLI

On this Page

Antigravity CLI Tutorial

Overview

Step-by-step

Next steps

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity CLI
>Using AGY CLI

Using AGY CLI

Settings

Antigravity CLI provides a flexible configuration system to customize workspace behavior, safety restrictions, editor preferences, visual style, and performance.

- Configuration File: Stored in a plain JSON file `~/.gemini/antigravity-cli/settings.json`.
- Settings Panel: Type `/config` or `/settings` to open a full-screen overlay menu listing all available options.
- Select a setting to open its list of options or a text input field.
- Immediately save your selection to the disk and return to the main list.
- Overrides: Certain settings can be overridden at launch via CLI flags (e.g., `--sandbox` or `--dangerously-skip-permissions`).
- The settings menu will display an indicator showing where the override came from (e.g., *Sandbox Mode on overridden by* `--sandbox`).
- You can still edit the persistent setting on disk, but the current session will enforce the command-line override until restarted.

Quick Tips

Action/Feature	Tip/Command

Auto-complete to file paths	`@` will trigger path suggestions
Clear Prompt	Type `esc esc` to clear your prompt box (when no streaming is active)
Terminal Commands	Use `!` at the start of your prompt to run terminal commands directly
Help	Type `?` to get help and list all slash commands
Reduce Noise from Tool Calls	Set verbosity to low in `/config` to minimize outputs from numerous tool calls
Manage Permissions	Control permissions via `/config` or `/permissions`

Go Back in Conversation	Use `/rewind` or `/undo` to rewind the conversation history
Fork Conversation	Use `/fork` to spin up a separate workspace and branch the conversation from an earlier point
Clear Conversation	Use `/clear` to clear the prompt and start a new conversation session
Resume Conversation	Use `/resume` to list and resume previous conversation logs
Auto-Save Resume	When you close the CLI, it automatically prints the exact command needed to resume that specific session

Keybindings

AGY CLI allows for custom keybindings. You can edit them by typing `/keybindings` or modifying the JSON file directly.

- File Location: `~/.gemini/antigravity-cli/keybindings.json`.
- Reset: To reset to default, delete the `keybindings.json` file.

Default Keybindings

Action/Command	Keys	Purpose
Clear TUI Screen	`ctrl+l`	Clear terminal output
Enter / Submit	`enter`	Submit prompts or choices
Escape / Cancel	`ctrl+c`, `esc`	Stop stream, close menus, or clear prompt

Exit CLI	`ctrl+d`	Terminate CLI TUI session
Suspend CLI	`ctrl+z`	Push CLI session to terminal background
Edit Command	`e`	Open editor to edit proposed terminal command
Confirm No	`n`	Decline terminal command execution
Confirm Yes	`y`	Approve terminal command execution
Open Editor	`ctrl+g`	Edit prompt inside your default shell editor

Paste Text	`ctrl+v`	Paste text from your clipboard
Redo Text Edit	`ctrl+shift+z`	Redo last undone text change
Undo Text Edit	`ctrl+_`, `ctrl+shift+-`	Undo last text change
Yank (Copy)	`ctrl+y`	Yank/copy selected text
Navigate Down	`down`	Scroll down in menu lists
Go to Bottom	`ctrl+end`	Jump TUI view directly to the bottom

Go to Top	`ctrl+home`	Jump TUI view directly to the top
Navigate Left	`left`	Move prompt cursor left
Page Down	`pgdown`, `shift+down`	Scroll TUI page down
Page Up	`pgup`, `shift+up`	Scroll TUI page up
Navigate Right	`right`	Move prompt cursor right
Tab / Focus	`tab`	Auto-complete choices or switch component focus

Navigate Up	`up`	Scroll up in menu lists

Insert Newline	`alt+enter`, `ctrl+j`, `shift+enter`	Add newline to prompt without submitting

You can map a single action to many keybindings in the JSON file. To disable keybindings, set the list to empty (e.g., `[]`). If the file is malformed, the CLI will use the valid parts and fall back to defaults for the broken actions.

warning

Important: Keybindings `cli.exit` and `cli.enter` cannot be disabled.

Tutorial

Features

On this Page

Using AGY CLI

Productskeyboard_arrow_down

Use Caseskeyboard_arrow_down

Pricing

Blog

Resourceskeyboard_arrow_down

Downloaddownload

Home

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity CLI

>Features

Antigravity CLI Features

Plugins

How Plugins Work Plugins are namespaced bundles that can contain skills, agents, rules, MCP servers, and hooks as a single deployable unit.

When you install a plugin, the CLI stages the files in your home directory under `~/.gemini/antigravity-cli/plugins/<plugin_name>/`. The Antigravity Agent automatically discovers and loads these staged customizations.

content_copy

```
~/.gemini/antigravity-cli/
├── plugins/
│   └── <plugin_name>/
│       ├── plugin.json         # Required marker file
│       ├── mcp_config.json     # Optional MCP server definitions
│       ├── hooks.json          # Optional event hooks definition
│       ├── skills/             # Optional skills
│       ├── agents/             # Optional subagents
│       └── rules/              # Optional rules
└── import_manifest.json        # Tracking manifest
```

Accessing Plugin Components Once staged and loaded, you can interact with the plugin components inside the CLI using slash commands.

Terminal Sandbox

The Terminal Sandbox is a lightweight security isolation mechanism that protects your host system from potentially destructive file manipulations or unauthorized outbound network requests when the agent executes local shell commands.

Rather than running heavy virtual machines or containers, the CLI leverages native operating system features (`nsjail` on Linux, `sandbox-exec` on macOS, and `AppContainer` on Windows) to enforce strict containment boundaries with zero startup overhead.

Configuration You can configure the sandbox behavior in your `settings.json` file (located at `~/.gemini/antigravity-cli/settings.json`):

json

content_copy

```
{
  "enableTerminalSandbox": true
}
```

- `enableTerminalSandbox` (boolean, default: `false`): Enables general execution containment barriers on all local agent processes.

Interactive Approvals When the agent proposes a terminal command that requires your confirmation, the CLI prompt adapts dynamically based on your settings:

- When the Sandbox is Enabled: The confirmation prompt will include a specific option to Yes, and run without sandbox restrictions if you need to temporarily bypass the containment boundary for a single trusted command.
- When the Sandbox is Disabled: The prompt will include an option to Yes, and run in sandbox if you want to force a specific, potentially risky command to execute within the safety boundary.

CLI Slash Commands Reference

The Antigravity CLI supports a variety of slash commands typed directly into the prompt box to manage conversations, configure settings, and inspect agent capabilities.

Core Slash Commands

Command	Category	Purpose

`/resume` *(alias `/switch`)*	Conversation	Open the conversation picker to resume or switch sessions.
`/rewind` *(alias `/undo`)*	Conversation	Roll back conversation history to a previous checkpoint.
`/rename <name>`	Conversation	Rename the active conversation thread for easier tracking.
`/permissions`	Configuration	Select agent autonomy level (`request-review`, `always-proceed`, or `strict`).
`/model`	Configuration	Select the default reasoning model (persists across sessions).
`/keybindings`	Configuration	Open the interactive keyboard shortcut editor.

`/statusline`	Configuration	Customize real-time indicators displayed in the CLI status bar.
`/tasks`	Tools & Monitoring	Monitor, view logs for, or terminate active background tasks.
`/skills`	Tools & Monitoring	Browse local and global encapsulated agent workflows.
`/mcp`	Tools & Monitoring	Open the panel to configure and manage Model Context Protocol servers.
`/open <path>`	Utility	Immediately open a file in your preferred external editor.
`/usage`	Utility	Open the inline interactive help manual inside the terminal.

`/logout`	Account	Log out of your Google session and clear cached credentials.

Advanced Customization via `settings.json`

For power users, several slash commands support deep customization via your `~/.gemini/antigravity-cli/settings.json` configuration:

- Fine-Grained Permissions: Instead of global levels, define specific allowed/denied commands:

content_copy

```
"permissions": {
  "allow": ["command(git)", "command(npm test)"],
  "deny": ["command(rm -rf)"]


}
```

- Custom Status Line & Window Titles: You can pipe live agent metadata (JSON format containing CWD, active model, token usage, state, etc.) directly into your own custom shell scripts to generate dynamic status bars or terminal window titles.

Subagents in Antigravity CLI

Antigravity CLI features an asynchronous subagents framework that allows the main agent to delegate parallel work, perform background research, and run system tests without blocking your active conversation.

What are Subagents? Subagents are independent, concurrent agent sessions designed to tackle specific background tasks in parallel with the main conversation.

- Purpose: The main agent automatically spawns subagents to perform background operations such as looking up documentation, running builds, or validating a fix.
- Capabilities: Subagents have full access to tools such as code search, file editing, terminal commands, and web searches to complete their assigned tasks.
- The main agent decides what tools and permissions subagents get, including whether they can use MCP tools and if they can write files.

Managing Agents: The `/agents` Panel

Antigravity CLI provides an interactive terminal UI to view, manage, and approve actions for running subagents.

- Access: Type `/agents` in the prompt to open the subagents panel.

- Overview: The panel shows a list of active and completed subagents, including surface-level details such as their status (running, done, killed, etc.) and the current step they are executing.

info

Selecting a subagent from the panel opens a full-screen detail view. This view shows the entirety of the subagent’s conversation, including its steps, thoughts, and tool execution logs.

Tool Confirmations & Approvals When a subagent wants to execute a tool that requires user permissions (such as running a local command or writing a file), it will surface the request. You can manage approvals in two ways:

1. Detail View Approvals

The Subagent Detail View features an interaction section containing all pending approvals, where you can selectively approve or deny requests.

lightbulb

Tip: Use the keyboard shortcut `ctrl+j` to "teleport" from the main conversation directly to the detailed view of the next subagent waiting for your approval.

1. Fast Path Alerts

To keep you in your flow, Antigravity CLI displays a Fast Path Alert directly above your prompt box when a subagent requests permission.

lightbulb

Tip: You can approve a pending subagent permission instantly using `ctrl+k` without ever having to switch away from the main conversation.

Using AGY CLI

Migration

On this Page

Antigravity CLI Features

Home

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity CLI
>Gemini Migration

Migrating from Gemini CLI

Convert your legacy configurations, import Gemini CLI extensions as native plugins, adapt custom skills paths, and reformat Model Context Protocol configurations.

Overview

Antigravity CLI preserves backward compatibility with the core developer-experience constructs popularized by Gemini CLI. To ensure a seamless upgrade, the CLI offers automatic onboarding conversion alongside explicit CLI migration command sequences.

First-launch onboarding

When you execute `agy` for the first time in an environment containing legacy configurations, the CLI automatically detects your existing profiles. An interactive checklist prompts you to choose which assets to migrate:

1. Auto-conversion: Select the extensions and global configurations you wish to convert.
2. Keyring storage: The CLI migrates your active session tokens securely into your operating system's native keyring storage.
3. Settings alignment: Default visual parameters and rendering buffers map automatically to your new settings profile.

info

Partial Parity: While we preserve support for workspace skills, rules, and MCP servers, certain customized terminal themes or experimental visual overlays from Gemini CLI may not be supported.

Converting extensions to plugins

Since Gemini CLI launched, the industry has standardized on the term plugins. You can manually convert your legacy Gemini extensions to native Antigravity plugins by executing:

bash

content_copy

```
agy plugin import gemini
```

This utility searches your legacy local directories, parses your extension manifests, and converts files into native layout blocks.

Expected import output

text

content_copy

```
[ok]   conductor-tools
       - skills     : skipped (none detected)
       - agents     : skipped (none detected)
       ✔ commands   : 4 legacy commands converted to skills
       - mcpServers : skipped (none detected)
[ok]   google-workspace
```

```
  ✔ skills      : 5 skills processed
  - agents      : skipped (none detected)
  ✔ commands    : 2 legacy commands converted to skills


  ✔ mcpServers : 1 server definition migrated to mcp_config.json
```

Context files and workspace rules

Both CLI platforms utilize identical workspace context rules. No modifications are needed to your existing rule documents:

- Workspace local context: The agent continues to parse and enforce rule constraints defined inside your active directory's `GEMINI.md` and `AGENTS.md` files.
- Global developer context: The agent automatically consults and enforces your global constraints located at `~/.gemini/GEMINI.md`.

Updated skills paths

While global shared skills remain in your user home directory, the target folder path for local workspace-specific skills has been updated.

Configuration	Gemini CLI	Antigravity CLI

Global shared path	`~/.gemini/skills/`	`~/.gemini/antigravity-cli/skills/`
Workspace project path	`.gemini/skills/`	`.agents/skills/`

warning

Action Required: If your project contains custom workspace skills defined in `.gemini/skills/`, you must manually rename or relocate the folder to `.agents/skills/` for the Antigravity agent to recognize them as active slash commands.

MCP config formatting changes

Antigravity CLI separates Model Context Protocol servers into dedicated, lightweight JSON profiles instead of nesting them inside your primary preferences configuration.

Directory mapping

- Legacy Gemini Config: Servers were declared inline within `~/.gemini/settings.json`.
- Antigravity CLI Config: Servers are defined inside a standalone `mcp_config.json` profile:
- Global servers: `~/.gemini/config/mcp_config.json`
- Workspace servers: `.agents/mcp_config.json`

Required schema updates

When manually migrating remote websocket or SSE server definitions, update the URI key parameter to match the current standard:

- Legacy schema keys: `url` or `httpUrl`
- Modern schema key: `serverUrl`

json

content_copy

```
{
  "mcpServers": {
    "remote-indexer": {
      "serverUrl": "https://mcp.internal.enterprise.com/sse",
      "env": {
        "AUTH_TOKEN": "secure_alpha_token"
      }
    }
  }
```

```
}
```

Next steps

Begin configuring your new visual parameters and troubleshooting any setup anomalies:

- Settings, Rendering & Keybindings: Customize keyboard hotkeys, themes, and screen buffers.
- Troubleshooting: Learn how to resolve authentication lockouts or path issues.
- CLI Reference: Access standard parameters lists and slash command mappings.

Features

Prompting & Interaction

On this Page

Migrating from Gemini CLI

Overview

First-launch onboarding

Converting extensions to plugins

Context files and workspace rules

Updated skills paths

MCP config formatting changes

Next steps

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Prompting & interaction

Master primary interaction patterns, multiline composition workflows, session interruption controls, and terminal media pasting.

The prompt box

Antigravity CLI features a sticky prompt panel positioned at the bottom of your terminal screen. This panel handles standard user entries, multiline scripts, and direct media pasting.

```
text

content_copy
──────────────────────────────────────────────────────────────────────
> Describe your next engineering task here...

──────────────────────────────────────────────────────────────────────
```

Submitting prompts

To initiate an agent turn, type your instruction into the prompt panel and press `Enter`. The agent immediately analyzes your current directory workspace, reads required configurations, and begins formulating an execution plan.

Interrupting active sessions

If the agent initiates an undesired task or loops during command execution, press `Esc` to immediately halt the session.

lightbulb

Universal Escape: The Esc key acts as a global escape hatch. Pressing Esc instantly cancels any active agent turn, closes overlay panels, and returns focus to a clean prompt box.

Multiline composition

For complex directives, structured test scenarios, or multi-paragraph instructions, use our built-in multiline features.

Shorthand newline insertions

- Standard: Press `Shift+Enter` or `ctrl+j` to insert a clean newline within your active prompt window without submitting.
- macOS Terminal Fallback: If using Apple Terminal (which does not forward `Shift+Enter` by default), press `Option+Enter`. Ensure you check Use Option as Meta key in your Terminal Preferences profile.
- Universal Slash Escape: Type a trailing backslash `\` at the end of your active line and press `Enter`. The CLI automatically removes the backslash and inserts a newline.

Editing prompts in `$EDITOR`

To draft or edit extensive prompt structures in your primary development editor:

1. Press `ctrl+g` inside the empty prompt panel.
2. The CLI launches your system's default text editor (such as `vim`, `nano`, or `code`, configured via `/config` or your environment's `$EDITOR` variable).
3. Draft your multi-line instruction inside the temporary editor buffer.
4. Save and exit the editor. The CLI automatically imports the edited buffer directly back into the terminal prompt.

Attaching media

Antigravity CLI supports pasting rich media formats directly from your system clipboard. Press `ctrl+v` (or native terminal paste) inside the prompt panel to attach screenshot mockups or video recordings.

Supported file types

- Images: PNG, JPEG, GIF, WebP, BMP, TIFF, and SVG.
- Videos: MP4, MOV, WebM, and AVI.

Next steps

After mastering interaction patterns, explore how the agent presents actions and requests verification:

- Reviewing Artifacts: Learn to inspect and manage file edits, plans, and test executions.
- Managing Conversations: Resume prior threads and fork active sessions.
- Background Tasks & Subagents: Monitor asynchronous background agents.

Migration

Reviewing Artifacts

On this Page

- Prompting & interaction
- The prompt box
- Multiline composition
- Attaching media
- Next steps

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

Reviewing artifacts

Audit generated code, review implementation proposals, attach line-level feedback comments, and verify visual media assets before applying edits to your local filesystem.

Collaboration and co-steering

An Artifact is a structured deliverable created by the agent to accomplish its task and communicate its progress and thinking to you. Artifacts include rich markdown outlines (such as Implementation Plans), code diffs, architecture diagrams, and visual media files.

As agents work with higher autonomy over longer periods, artifacts enable asynchronous collaboration. You do not need to carefully monitor every individual tool execution synchronously. Instead, you review high-level deliverables at key milestones.

Because autonomous agents can occasionally go off-course or hallucinate solutions, the artifact workflow serves as a critical interactive co-steering mechanism. Depending on your configuration, the agent will pause at intermediate milestones, allowing you to inspect proposed plans or code edits, provide inline comments, and redirect the agent before any changes are physically written to your local filesystem.

The TUI partitions these assets into two interactive layers:

- The Artifact Picker Overlay: A high-level checklist menu containing review status markers, quick preview toggles, and collapsible folders.
- The Artifact Detail Viewer: A full-screen code audit interface supporting inline commenting, syntax highlighting, and diagram scaling.

Overview of /artifact

When the agent produces or modifies files, a notification updates in your TUI status bar (`/artifact to review`). Press `ctrl+r` inside the prompt box to open the full-screen Artifact Picker Panel.

text

content_copy

10 artifacts · /artifact to review
_____
_____
_____
>
_____
_____
_____
Action required (10 left)
› □ new release_notes.md open approve reject
  utils.py
  □ new performance_report.md
  api_client.py
  config_manager.py

□ new user_guide.md
run_tests.py
data_processor.py
□ new system_architecture.md
□ new project_overview.md

```
Keyboard: ↑/↓ Navigate  y/n Approve/reject  shift+a Approve all  p Preview  esc Done
```

Interaction keybindings

Audit the file checklist using the following dedicated panel controls:

Key	TUI Command	Action Behavior
`↑` / `↓`	`nav.scroll_line`	Scrolls highlighted selections up and down through the list of entries.
`h` / `l`	`nav.switch_button`	Focuses and toggles between inline row buttons: open, approve, and reject (Left/Right arrows also supported).

`p`	`confirm.preview`	Toggles a quick inline file preview. This opens a 12-line truncated and indented code block preview directly under the selected row.
`y`	`confirm.approve`	Instantly approves the highlighted file. The status marker updates to a green checkmark (`✓ approved`).
`n`	`confirm.reject`	Instantly rejects the highlighted file. The status marker updates to a red cross (`✗ rejected`).
`Shift+A`	`confirm.approve_all`	Bulk-approves all pending actionable files in one action.
`Shift+R`	`confirm.reject_all`	Bulk-rejects all pending actionable files in one action.
`Enter`	`nav.confirm`	Executes the active focused button. If the `open` button is focused, it launches the full-screen Detail Viewer.

`Esc`	`nav.escape`	Saves your active review state, submits approvals/rejections back to the agent thread, and returns focus to the prompt box.

Code files vs visual media

To organize workspace assets, the picker separates files by format types:

- Actionable Code Files: Standard programming codes, configs, and plan markdowns that require explicit approvals.
- Collapsible Media Drawer: Visual asset files (such as PNG, JPG, WebP, SVG, MP4, or WebM media) are grouped into a dedicated "Media" drawer header.
- Highlight the Media header row and press `Enter` to expand or collapse the drawer list.
- Highlight a specific media item and press `Enter` to open the file inside your operating system's native media viewer.

Viewing an artifact

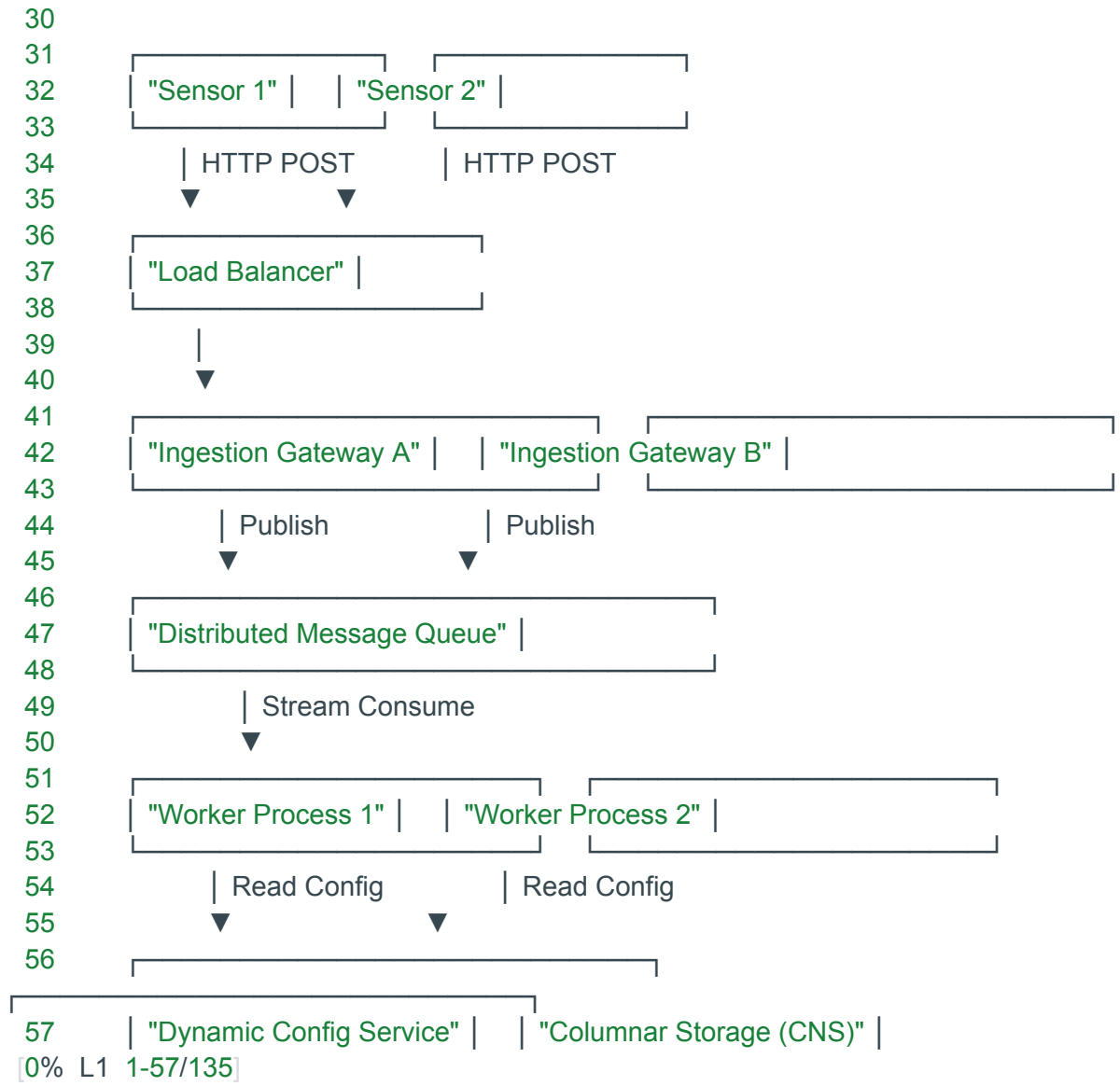
To launch a close audit of a file's code structure or proposed logic, select `open` (or press `Enter` directly on a highlighted code row) to open the Artifact Detail Viewer.

text

content_copy

implementation_plan.md

```
>  1  Implementation Plan: Alpha-Centauri Telemetry Scaling Engine
   2
   3  This document provides a highly detailed, step-by-step engineering implementation plan to upgrade the Alpha-Centauri
   4  telemetry ingestion pipeline. It outlines current gaps, proposed architecture improvements, execution timelines, risks,
   5  and verification procedures.
   6  ———————
   7  ## 1. Executive Summary
   8
   9  As sensor deployments scale from 100 to 10,000 active nodes, the existing synchronous Python-based ingestion system (
  10   data_processor.py ) faces critical CPU and write latency bottlenecks.
  11
  12   This implementation plan details the migration to an asynchronous, remote-buffered pipeline utilizing distributed
  13   message
  14   queues, multi-threaded worker pools, and an optimized column-oriented storage layer.
  15   ———————
  16   ## 2. Current Architecture vs. Target Architecture
  17
  18   ### Gap Analysis
  19
  20    Feature    | Existing (v1.2)   | Target (v2.0)     | Gap to Resolve
  21   -----------|------------------|------------------|-------------------------
  22    Concurrency| Sync, 1-thread    | Async, concurrent | Cannot scale peak bursts
  23    Buffer     | None (direct API) | Message Queue     | Outage data loss
  24    Storage    | Flat JSON         | Columnar CNS      | Slow queries, high consumption
  25    Config     | Load on launch    | Dynamic polling   | Requires restarts to update
  26
  27   ### Architectural Schema
  28
  29      Ingestion Layer │ Buffering Layer │ Processing Layer │ Storage Layer
```



0% L1 1-57/135

↑/↓ scroll · pgup/pgdown page · shift+g bottom · g top · c comment · m raw mermaid · ctrl+=/ctrl+- zoom 100%

I hide lines · esc close

Auditing & navigation

- Scrolling: Scroll page-by-page or line-by-line using `j`/`k` (or standard arrow keys).
- Boundary Jump: Press `g` to jump to the top of the file, and `Shift+G` to jump directly to the bottom.
- Toggle Gutter: Press `l` to toggle the line number gutter on and off for a cleaner presentation of the raw code.

Granular line commenting

If a specific block of code requires correction:

1. Navigate and position your cursor on the target line.
2. Press `c` to open an inline, multi-line text editor buffer attached directly to that line.
3. Draft your descriptive feedback, and press `Esc` to save and submit the comment. The line is updated with a visual comment indicator (💬).
4. To delete your active feedback, position your cursor on the commented line and press `d`.

Custom Mermaid diagram rendering

If the active document contains structured system flowcharts, database relationships, or architectural layouts:

- Cycle Render Modes (`m`): Press `m` to cycle visual rendering modes:

- Kitty Graphics Image: Renders diagrams natively as inline graphics within Kitty-compatible terminal emulators.
- ASCII Box Art (Default): Renders diagrams as clean, high-performance text art compatible with all shells.
- Raw Code: Shows the raw markdown code block fences.
- Zooming Graphics: When Kitty graphics image mode is active, press `ctrl+=` to zoom in and scale up the image, and `ctrl+-` to zoom out.

Press `Esc` to close the Detail Viewer and return back to the primary picker checklist.

Next steps

Configure settings preferences and review agent autonomy parameters:

- Managing Conversations: Resume prior sessions and fork branches.
- Settings, Rendering & Keybindings: Customize keyboard hotkeys and visual buffers.
- Permissions & Sandbox: Configure security parameters and containment lists.

Managing Conversations

On this Page

- Reviewing artifacts
- Collaboration and co-steering
- Overview of /artifact
- Viewing an artifact
- Next steps

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

Managing conversations

Resume prior development threads, scope active histories to local workspaces, and fork conversations to experiment with alternate architectures.

Workspace scoping

To maintain context hygiene, Antigravity CLI scopes conversation histories directly to your current working directory. When you launch `agy` from a specific directory, the agent only displays and resume sessions associated with that specific local repository or subdirectory.

This prevents context pollution, ensuring that the agent's semantic memory and token limits remain focused solely on the relevant codebase.

Resuming sessions

You can return to a prior conversation to continue an implementation or refine an existing solution.

Resuming via the TUI session picker

To search and load previous conversations within your active terminal screen:

1. Type `/resume` in the prompt box and press `Enter`.
2. The interactive Conversation Picker overlay opens.
3. Start typing keyword terms to filter conversations by description or ID.
4. Use `↑`/`↓` to navigate the list, and `←`/`→` to page through older records.
5. Press `Enter` to resume the selected conversation. Press `Esc` to cancel and return to the prompt.

text

content_copy

```
  CLI    Antigravity   (tab to cycle)

  Conversations
  Type to search...
> implement-auth-pipeline                          4 steps   3h ago
  refactor-db-connection-pool                      7 steps   5h ago
  add-unit-tests-for-parser                      2 steps   1d ago
  fix-socket-connection-timeout-error              14 steps   2d ago
  update-project-dependencies                      5 steps   3d ago
  optimize-image-compression-algorithm                9 steps   4d ago
  draft-release-notes-v2.1.0                     3 steps   May 23
  clean-up-obsolete-cache-files                    6 steps   May 22
  integrate-payment-gateway-sdk                    19 steps   May 20
  fix-layout-alignment-in-navbar                 8 steps   May 19
  [1-10 of 75 items]
```

Keyboard ↑/↓ Navigate ←/→ Page enter Select f2 Rename tab Switch esc Done

Importing conversations from Antigravity 2.0

If you are utilizing the public version of Antigravity CLI, you can import and resume active threads initiated within the Antigravity 2.0 desktop visual editor:

1. Type `/resume` in the prompt panel and press `Enter` to open the picker.
2. Press `Tab` to cycle between the CLI tab (local TUI conversations) and the Antigravity tab (Antigravity 2.0 desktop conversations).
3. Highlight your target desktop conversation using `↑/↓` and press `Enter`.
4. The TUI displays a confirmation prompt. Press `Enter` (or `y`) to confirm the import.
5. The CLI duplicates the desktop conversation history, context, and trajectories into your terminal session, allowing you to continue the workflow seamlessly.

text

content_copy

```
CLI  Antigravity  (tab to cycle)

Conversations
Type to search...
a1b2c3d4-e5f6-7890-abcd-ef1234567890                loading…  May 23
f9e8d7c6-b5a4-3210-fedc-ba9876543210                loading…  May 16
> Design New Analytics Dashboard Layout [Import this? (y/n)] 12 steps  Apr 30
Implement Realtime Graph Plotter                    15 steps  Apr 30
fix-visual-flashes-on-load                          2 steps  Apr 30
add-collapsible-menu-sidebar                        4 steps  Apr 29
refactor-utility-helper-methods                     6 steps  Apr 28
```

Verify Webpack Configuration Output 5 steps Apr 28
[1-10 of 13 items]

Keyboard ↑/↓ Navigate enter Select tab Switch Tab esc Go back / Clear search

Quick resume via the command line

To instantly resume the single most recent session in your active workspace without entering the picker, launch the executable with the `--continue` flag:

bash

content_copy

```
agy --continue
```

To load a specific session directly from your shell, pass the target UUID:

bash

content_copy

```
agy --conversation 9a8b7c6d-5e4f-3a2b-1c0d-ef1234567890
```

Branching with \`/fork\`

When engineering a complex feature, you may want to explore multiple design alternatives without losing your progress. The `/fork` command enables safe, parallel experimentation.

```
text

content_copy

/fork
```

(Alias: `/branch`)

The `/fork` command clones your entire conversation history up to the current turn into a new, independent session.

Forking workflow

1. Type `/fork` inside the prompt panel and press `Enter`.

2. The CLI allocates a new unique session ID and duplicates your existing workspace state and agent thread.
3. Your active terminal switches immediately to the new branch.
4. If the experiment fails, run `/resume` to restore your original, stable conversation branch.

lightbulb

Branching Filesystems: Forking clones the *conversation thread*, not your local git checkout. To fully isolate files during parallel forks, use git branches or stash local changes before testing contrasting approaches.

Next steps

Explore how the agent handles complex, asynchronous operations and parallel tasks:

- Background Tasks & Subagents: Monitor subagents and handle fast-path approvals.
- Settings, Rendering & Keybindings: Configure rendering buffers and override JSON preferences.
- Permissions & Sandbox: Manage security profiles and system command lists.

Reviewing Artifacts

Background Tasks & Subagents

On this Page

Managing conversations

Workspace scoping

Resuming sessions

Branching with \`/fork\`

Next steps

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Background tasks & subagents

Delegate slow builds, multi-file code generation, and research sweeps to parallel background agents while maintaining your active programming flow.

Asynchronous execution model

To maximize developer velocity, Antigravity CLI leverages a multi-threaded asynchronous execution architecture. Instead of locking your terminal session during long-running builds, massive codebase search sweeps, or complex multi-file edits, the primary agent delegates these operations to parallel Subagents or background Tasks.

This delegation model ensures you never have to wait on high-latency AI processes. You can continue drafting code, submitting prompts, or inspecting files while multiple autonomous background threads execute validation tasks in parallel.

Managing agents: The `/agents` panel

The active agent-hierarchy is fully transparent and manageable through an interactive terminal interface.

Opening the panel

Type `/agents` in the prompt and press `Enter` to open the interactive Agent Manager Panel.

Panel overview

The panel displays a live checklist of all active, completed, killed, or failed background agents:

- Identifier: The unique target subagent ID.
- Role: The specialized role of the agent (such as "Codebase Researcher" or "Database Debugger").
- State: Live status indicators (running, done, killed, or error).
- Step: A real-time summary of the tool or reasoning step currently being executed.

Deep-dive monitoring

To inspect the inner reasoning, thoughts, and logs of a specific background agent:

1. Open the `/agents` panel and highlight the target agent using `↑`/`↓`.
2. Press `Enter` to open the Subagent Detail View.
3. This full-screen view reveals the subagent's entire reasoning log, including its private internal thoughts, tool calls, and execution outputs.
4. Press `Esc` to exit and return to the main Agent Manager list.

Monitoring background tasks with `/tasks`

For non-agentic background operations, such as direct shell commands, testing suites, or simple background queries initiated via `/btw`, use the `/tasks` command.

text

content_copy

```
/tasks
```

The tasks tracking list lets you:

- Track standard non-interactive background processes.
- Select a task using `↑/↓` and press `Enter` to view stdout logs.
- Terminate runaway terminal processes safely.

Keyboard ergonomics

To reduce context-switching friction when subagents require manual interaction or tool authorizations, Antigravity CLI integrates high-efficiency shortcut paths.

Detailed "Teleport" navigation (`Ctrl+J`)

When a subagent encounters a tool requiring approval (e.g. writing a file or running a database migration), a status bar notification blinks.

- Press `Alt+J` inside the main prompt panel to instantly "teleport" from your current conversation directly into the Detail View of the next subagent awaiting your approval.
- Confirm or reject the action, and press `Esc` to teleport back to your primary thread.

"Fast-Path" confirmations (`Ctrl+K`)

To authorize an agent action instantly without leaving your active workspace:

1. Look at the inline status notification displayed right above your active prompt box. It summarizes the pending action (e.g., `Subagent 12 asks to run "npm test"`).
2. Press `Ctrl+K` to instantly approve the pending fast-path action without switching panels or opening overlays.

Next steps

Configure the visual shell behavior and customize your configuration profiles:

- Settings, Rendering & Keybindings: Customize key maps, buffering, and JSON rules.
- Permissions & Sandbox: Enforce security containment rings on background processes.
- Plugins & Skills: Create your own custom skills and slash commands.

Managing Conversations

Permissions

On this Page

Background tasks & subagents

Asynchronous execution model

Managing agents: The `/agents` panel

Deep-dive monitoring

Monitoring background tasks with `/tasks`

Keyboard ergonomics

Next steps

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

Permissions

Secure your local workstation, restrict absolute file paths, configure custom allow/deny/ask policies, and manage interactive approvals.

Fine-grained permissions

To secure your workstation while enabling autonomous workflows, Antigravity CLI integrates a robust Fine-Grained Permissions Engine. Every sensitive operation the agent performs is represented as a permission resource formatted as `action(target)`.

Permissions are evaluated across three distinct access lists configured inside your global settings:

text

content_copy

~/.gemini/antigravity-cli/settings.json

- `deny`: The action is blocked immediately.
- `ask`: The agent pauses and prompts for your explicit approval before proceeding.
- `allow`: The action is auto-approved without prompting.

warning

Precedence Rule: Conflicting rules are strictly evaluated in priority order: **Deny > Ask > Allow**. For example, if you configure `command(*)` in your `ask` list and `command(git)` in your `allow` list, the `ask` rule takes precedence and prompts before every command.

Supported actions & matching rules

Fine-grained permissions follow a standard schema pattern:

text

content_copy

action(target)

The supported actions, target format specifications, and matching algorithms are:

Action	Target Format	Matching Behavior	Default Fallback		
`read_file`	`read_file(/path)`, `read_file(dir)`, or `read_file(*)`	Matches absolute paths or paths relative to workspace roots. Grants recursive read access to all contained files/folders. `read_file(*)` matches all files on the system.	Ask (Auto-allowed in workspace)		
`write_file`	`write_file(/path)` or `write_file(*)`	Same as `read_file`. Implicitly grants `read_file` for the exact same target path.	Ask (Auto-allowed in workspace)		

`read_url`	`read_url(domain)` or `read_url(*)`	Matches hostnames and subdomains (e.g., `google.com` covers `mail.google.com`). Ignores URL path segments. `read_url(*)` matches any domain.	Ask		
`execute_url`	`execute_url(domain)` or `execute_url(*)`	Actuating on web elements (clicking, typing) or driving interactive browser workflows on a domain.	Ask		
`command`	`command(prefix)`, `command(regex)`, or `command(*)`	Matches commands by exact word/token prefix. Each whitespace-separated token is evaluated as an anchored regular expression (`^(?:pattern)$`). E.g., `command(npm run (build\	lint\	test))`matches`npm run buildand npm run test`.	Ask

`unsandboxed`	`unsandboxed(prefix)` or `unsandboxed(*)`	Matches commands by exact word/token prefix. Commands matching this grant will be executed outside of container isolation (only applicable when terminal sandboxing is enabled).	Ask	
`mcp`	`mcp(server/tool)` or `mcp(*)`	Matches exact MCP tools or all tools on a specified server (applies to local `mcpl` servers and remote connections). `mcp(*)` matches any tool.	Ask	

Global wildcard syntax

Across all supported action types, passing the global wildcard `*` (such as `read_file(*)`, `command(*)`, `mcp(*)`) matches all targets within that entire action namespace.

Implicit permission rules

- Write implies Read: Allowing `write_file` on a path automatically grants `read_file` on that path.
- Deny Read implies Deny Write: Denying `read_file` on a path immediately blocks `write_file` on that path.

Cross-platform path normalization

Antigravity ensures your permission rules work flawlessly whether you are developing on macOS, Linux, or Windows. On macOS and Linux, paths use standard forward slashes (`/`). On Windows, Antigravity automatically normalizes paths prior to rule evaluation by stripping drive letters (e.g., `C:`) and converting all backslashes (`\`) to forward slashes (`/`).

Default system behaviors & guardrails

When an action is not explicitly listed in your `allow`, `deny`, or `ask` lists, the system falls back to secure system defaults:

1. Workspaces are Auto-Allowed: In standard operation, reading and writing files inside your active project directory is automatically allowed.
2. Web Browsing Defaults to Ask: Actions for `read_url` and `execute_url` default to Ask. Before the agent navigates to or actuates on any web page, it will pause and prompt for your approval unless an allow rule is configured.

3. Unconfigured Actions Default to Ask: All other unconfigured actions (`command`, `mcp`, `execute_url`, non-workspace files) default to Ask.

Interactive permission prompts

When the agent encounters an operation requiring approval (Ask mode), an interactive prompt card appears in your TUI.

Before confirming Allow for file, URL, or MCP permissions, you can directly edit the target string in the prompt card to expand the granted scope (e.g., broadening a single file request like `/project/file.txt` to the parent directory `/project`). The CLI validates that your edited target safely covers the operation and applies the expanded grant for the remainder of the turn, preventing repeated prompts for related operations. *(Note: Scope editing is not supported for terminal commands).*

Configuration examples

Add these rules to your `~/.gemini/antigravity-cli/settings.json` file:

```
json

content_copy
{
  "permissions": {
    "allow": [
      "command(git)",
      "command(npm run (build|lint|test))",
      "unsandboxed(git push)",
      "read_file(/var/log/app)",
      "write_file(src/)",
      "read_url(google.com)",
      "mcp(linter/*)"
    ],
    "deny": [
      "command(rm -rf)",
      "command(curl .*)",
      "command(sudo)",
      "write_file(.git/)",
      "write_file(/home/user/.ssh)"
    ],
    "ask": [
      "command(*)",
      "execute_url(aws.amazon.com)",
      "mcp(sql/execute_mutation)"
    ]
  }
}
```

See also

- Sandbox Customization: Enforce OS-level container isolation boundaries.
- Plugins & Skills: Create your own custom skills slash commands.

- Settings, Rendering & Keybindings: Customize keyboard hotkeys and buffers.

Background Tasks & Subagents

Sandbox

On this Page

Permissions

Fine-grained permissions

Supported actions & matching rules

Default system behaviors & guardrails

Interactive permission prompts

Configuration examples

See also

Home

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity CLI
>Agent Capabilities
>Sandbox

Sandbox

Enforce native operating system process isolation, manage execution containment boundaries, and protect your local workstation.

The security model

Because autonomous development agents run local terminal commands, edit source codes, and execute tests directly in your workspace, maintaining a secure workstation environment is critical. Antigravity CLI integrates a native Terminal Sandbox to restrict destructive shell operations or unauthorized remote network calls.

Native OS containment

Unlike heavy virtual containers or isolated virtual machines that slow down execution speeds, Antigravity uses lightweight, native operating system kernel utilities to create secure process rings with zero execution overhead:

Operating System	Sandboxing Utility	Security Characteristics
Linux	`nsjail`	Open-source process isolator utilizing kernel namespaces and cgroups to confine CPU, memory, and path visibility.

macOS	`sandbox-exec`	Native system tool enforcing policy profiles that restrict absolute filesystem access and raw TCP queries.
Windows	`AppContainer`	Desktop security containment ring isolating filesystem permissions and registry visibility.

Activating the sandbox

You configure the sandbox directly inside your global preferences:

text

content_copy

~/.gemini/antigravity-cli/settings.json

Sandbox configurations

Add the sandboxing toggle to your settings profile:

```
json

content_copy
{
  "enableTerminalSandbox": true
}
```

- `enableTerminalSandbox` (boolean, default: `false`): Restricts all local execution commands launched by agents to OS containment rings.

Interactive approvals with sandbox

When the agent attempts to run a terminal tool or shell command, the TUI prompt block adapts dynamically based on your sandboxing state:

- When Sandbox is Enabled: The prompt panel offers a temporary escape option:

```
content_copy
Do you want to proceed?
 1. Yes
 2. Yes, and run without sandbox restrictions
 3. No
```

Choosing Option 2 bypasses the containment barrier exclusively for that single execution run.

- When Sandbox is Disabled: The prompt lets you force containment for a risky command:

```
content_copy
Do you want to proceed?
 1. Yes
 2. Yes, and run in sandbox

 3. No
```

See also

- Permissions Engine: Configure fine-grained allow/deny policy rules.
- Plugins & Skills: Create your own custom skills slash commands.
- Settings, Rendering & Keybindings: Customize keyboard hotkeys and buffers.

Settings, Rendering & Keybindings

On this Page

- Sandbox
- The security model
- Activating the sandbox
- Interactive approvals with sandbox
- See also

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

Overview

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

Sandbox

Enforce native operating system process isolation, manage execution containment boundaries, and protect your local workstation.

The security model

Because autonomous development agents run local terminal commands, edit source codes, and execute tests directly in your workspace, maintaining a secure workstation environment is critical. Antigravity CLI integrates a native Terminal Sandbox to restrict destructive shell operations or unauthorized remote network calls.

Native OS containment

Unlike heavy virtual containers or isolated virtual machines that slow down execution speeds, Antigravity uses lightweight, native operating system kernel utilities to create secure process rings with zero execution overhead:

Operating System	Sandboxing Utility	Security Characteristics
Linux	`nsjail`	Open-source process isolator utilizing kernel namespaces and cgroups to confine CPU, memory, and path visibility.
macOS	`sandbox-exec`	Native system tool enforcing policy profiles that restrict absolute filesystem access and raw TCP queries.
Windows	`AppContainer`	Desktop security containment ring isolating filesystem permissions and registry visibility.

Activating the sandbox

You configure the sandbox directly inside your global preferences:

```
text
```

content_copy

```
~/.gemini/antigravity-cli/settings.json
```

Sandbox configurations

Add the sandboxing toggle to your settings profile:

json

content_copy

```
{
  "enableTerminalSandbox": true
}
```

- `enableTerminalSandbox` (boolean, default: `false`): Restricts all local execution commands launched by agents to OS containment rings.

Interactive approvals with sandbox

When the agent attempts to run a terminal tool or shell command, the TUI prompt block adapts dynamically based on your sandboxing state:

- When Sandbox is Enabled: The prompt panel offers a temporary escape option:

```
content_copy
Do you want to proceed?
 1. Yes
 2. Yes, and run without sandbox restrictions


 3. No
```

Choosing Option 2 bypasses the containment barrier exclusively for that single execution run.

- When Sandbox is Disabled: The prompt lets you force containment for a risky command:

```
content_copy
Do you want to proceed?
 1. Yes
 2. Yes, and run in sandbox


 3. No
```

See also

- Permissions Engine: Configure fine-grained allow/deny policy rules.
- Plugins & Skills: Create your own custom skills slash commands.
- Settings, Rendering & Keybindings: Customize keyboard hotkeys and buffers.

Permissions

Settings, Rendering & Keybindings

On this Page

Sandbox

The security model

Activating the sandbox

Interactive approvals with sandbox

See also

Productskeyboard_arrow_down

Use Caseskeyboard_arrow_down

Pricing

Blog

Resourceskeyboard_arrow_down

Downloaddownload

Home

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

Overview

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity CLI
>Settings
>Overview

Settings, rendering & keybindings

Configure persistent preferences, customize keyboard shortcuts, toggle terminal display buffers, and manage runtime CLI parameter overrides.

Setting up preferences

Antigravity CLI stores user preferences in a minimal, forward-compatible JSON configuration profile.

Configuration file location

The persistent settings are saved in a plain JSON format:

text

content_copy

~/.gemini/antigravity-cli/settings.json

The CLI leverages sparse persistence by writing only values to disk that differ from their system defaults. This keeps your configuration file clean, minimal, and fully forward-compatible with future updates.

The interactive settings panel

To edit settings directly inside your active terminal session without opening raw JSON files:

1. Type `/config` (or its alias `/settings`) inside the prompt panel and press `Enter`.
2. The full-screen Settings Editor Overlay opens.
3. Navigate between available options using `↑`/`↓`.
4. Press `Enter` on a highlighted parameter to toggle its state or open a text insertion field.
5. Press `Esc` to save your modifications and close the editor.

Command-line overrides

You can temporarily override persistent preferences for individual terminal sessions using CLI command flags:

```
bash

content_copy

agy --sandbox=false --notifications=false
```

When an override flag is active, the interactive `/config` menu displays a warning indicator alongside the modified setting:

```
text

content_copy

! Tool Permission  strict (overridden by command flag)
```

You can still edit the persistent value on disk during these sessions, but the CLI enforces the active runtime flag override until you close the session.

Visual rendering modes

The TUI operates in one of two visual rendering modes depending on your terminal capability and connection latency.

Alt-screen mode (`always`)

This mode opens a dedicated display screen using the terminal's alternate buffer, creating an immersive, standalone app interface.

- Key features: Integrated scrollback, mouse-wheel scrolling support, custom rendered scrollbar, and clean terminal state restoration on exit.
- Best used for: Standard local development sessions in advanced terminal emulators (such as iTerm2, Ghostty, or WezTerm).

Inline mode (`never`)

This mode renders output sequentially directly within your terminal's standard stdout pipeline.

- Key features: Preserves entire session history inside your emulator's native scrollback buffer, does not capture mouse inputs, and works seamlessly alongside standard command outputs.
- Best used for: Remote SSH terminals, terminal multiplexers like `tmux` or `screen`, and low-bandwidth remote sessions.

info

Adaptive Rendering: Setting Alt-Screen mode to `default` allows the TUI to automatically detect your environment. It defaults to Alt-Screen on advanced local shells and degrades to Inline mode when running over SSH or in non-interactive sessions.

Custom status lines & terminal titles

For advanced TUI environment integrations, you can toggle active metrics or deploy custom scripts to generate dynamic status bars and modify your terminal window titles:

- Status Line Customization: Learn how to manage the status indicator panel and construct custom formatted status line shell scripts.
- Terminal Title Customization: Learn how to toggle window title outputs and pipe live agent states into your window headers.

Keybindings configuration

You can customize almost all keyboard shortcuts in the TUI by mapping keys to specific workspace commands.

Keybindings file location

Custom maps are stored alongside your primary settings profile:

text

content_copy

~/.gemini/antigravity-cli/keybindings.json

Format and customization

The JSON structure maps a single TUI command action to an array of hotkey sequences:

```
json

content_copy
{
  "cli.clear_screen": [
    "ctrl+l"
  ],
  "prompt.insert_newline": [
    "shift+enter",
    "ctrl+j"
  ],
  "edit.open_editor": [
    "ctrl+g"
  ]

}
```

To completely disable a default hotkey, map its action to an empty array `[]`. If your JSON schema is malformed or invalid, the CLI falls back to system defaults for those specific actions and loads the remaining valid mappings.

```
warning
```

Protected Keys: Crucial navigation shortcuts like `cli.exit` (`Ctrl+D` / `Ctrl+C`) and `cli.enter` (`Enter`) are protected by the system and cannot be disabled.

Restoring defaults

To revert all keys back to system defaults, simply delete the keybindings profile:

bash

content_copy

```
rm ~/.gemini/antigravity-cli/keybindings.json
```

Next steps

Now that you have configured your environment, review security controls and extensibility options:

- Permissions & Sandbox: Manage secure execution containment boundaries.
- Plugins & Skills: Create your own custom skills and import legacy plugins.
- CLI Reference: Access quick reference sheets listing all configuration options, commands, and default key maps.

Sandbox

AI Credits

On this Page

- Settings, rendering & keybindings
- Setting up preferences
- Command-line overrides
- Visual rendering modes
- Custom status lines & terminal titles
- Keybindings configuration
- Next steps

Productskeyboard_arrow_down

Use Caseskeyboard_arrow_down

Pricing

Blog

Resourceskeyboard_arrow_down

Downloaddownload

Home

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

Overview

AI Credits

Customizations

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity CLI
>AI Credits

Managing AI Credits & Quotas

The Antigravity CLI integrates with your subscription to monitor and manage your AI Premium credits and usage quotas.

For a detailed explanation of baseline quotas, how credits are consumed, and plan eligibility, please refer to the main Plans page.

Quota Tracking

You can monitor your active quota and credit consumption directly inside the CLI:

- Statusline Indicator: The right side of the CLI statusline displays your remaining credit count (e.g., `AI Credits: 42`).
- Low Quota Alert: When your remaining AI credits drop below the warning threshold, the statusline indicator highlights to warn you that your limits are near.

Slash Commands & Managing Balance

You can query your credits or buy additional quota directly from the CLI:

- Query Balance: Type `/credits` in the prompt to open the dedicated credits panel. This panel displays your detailed credit usage statistics.
- Managing Credits: You can easily purchase AI credits or upgrade your subscription, which opens a panel containing direct pricing and subscription portal links.

Settings Configuration

To control when and how your AI credits are used, you can toggle credit settings:

- Use AI Credits Option: Run `/config` or `/settings` to open the CLI settings panel. Set the Use G1 Credits field to on to allow the CLI to use your personal credits when plan

quotas are exhausted, or set it to off to restrict fallback billing. (To learn more, see the Plans overages section).

Settings, Rendering & Keybindings

Plugins & Skills

On this Page

Managing AI Credits & Quotas

Quota Tracking

Slash Commands & Managing Balance

Settings Configuration

Blog

Resourceskeyboard_arrow_down

Downloaddownload

Home

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

Overview

AI Credits

Customizations

Plugins & Skills

Status Line

Window Title

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity CLI
>Customizations
>Plugins & Skills

Plugins & skills

Extend agent capabilities, install third-party extension bundles, package custom workflow skills, and interface with Model Context Protocol (MCP) servers.

The extensibility model

Antigravity CLI is designed for limitless customization. You can augment the shared agent harness by installing structured package modules called Plugins or creating localized markdown blueprints called Skills.

These customizations allow agents to access specialized proprietary commands, invoke domain-specific subagents, and consult customized style constraints.

Antigravity plugins

Plugins are namespaced bundles that package custom skills, background subagents, linting rules, Model Context Protocol definitions, and event hooks into a single deployable asset.

Plugin filesystem structure

When you install or import a plugin, the CLI stages the bundle files within your global configuration path:

text

content_copy

```
~/.gemini/antigravity-cli/plugins/<plugin_name>/
```

A compliant plugin contains the following layout:

text

content_copy

```
~/.gemini/antigravity-cli/plugins/<plugin_name>/
├── plugin.json              # Required package marker file
├── mcp_config.json          # Optional Model Context Protocol servers
├── hooks.json               # Optional pre/post tool event hooks
├── skills/                  # Optional specialized skills directory
├── agents/                  # Optional subagent definition templates
└── rules/                   # Optional custom codebase rules files
```

Managing plugins via CLI subcommands

The CLI exposes a `plugin` (or plural `plugins`) subcommand pipeline to manage your extensions:

- List installed plugins: Show active packages and their loaded components.

content_copy

```
agy plugin list
```

- Install a local or remote plugin: Stage a package directory into your local profile.

content_copy

```
agy plugin install /path/to/local/plugin
```

- Disable/Enable a plugin: Suspend a plugin's tools without deleting its assets.

content_copy

```
agy plugin disable <plugin_name>
```

```
agy plugin enable <plugin_name>
```

- Uninstall a plugin: Purge the package directory and clean up registries.

content_copy

```
agy plugin uninstall <plugin_name>
```

Agent skills

Skills are declarative, human-readable markdown files that outline explicit instruction protocols, scripts, and target resources for specialized engineering tasks.

Once registered, Skills convert automatically into slash commands inside the TUI, allowing you to invoke them manually (e.g., typing `/refactor-ui`).

Creating local workspace skills

To deploy workspace-specific skills that stay with your git repository:

1. Create a directory named `.agents/skills/` at your project root.
2. Inside, draft a markdown file with a `.md` extension (such as `format-tests.md`).
3. Define the skill's Frontmatter metadata:

content_copy

```
---
name: format-tests
description: Standardize and re-format Python unittest assertions


---
```

1. Below the metadata, write explicit instructions for the agent. When you run `agy` in this directory, the skill is compiled, and `/format-tests` becomes available in the prompt box.

Sharing global skills

To share skills across all workspaces on your workstation, place the target markdown files inside your global configuration path:

text

content_copy

```
~/.gemini/antigravity-cli/skills/
```

Any markdown skill in this directory is automatically imported as a global slash command whenever you launch `agy` in any directory.

Managing hooks

Hooks intercept agent actions right before or immediately after execution. They are useful for running automated pre-flight checks or post-generation formats (such as running `prettier` after writing files).

Hooks are defined inside a plugin's `hooks.json` or configured inside your primary `settings.json` file. You can inspect all loaded and active hooks inside the TUI by typing:

text

content_copy

/hooks

Model Context Protocol (MCP)

Model Context Protocol is an open standard enabling foundation models to interface securely with local APIs, file parsers, and custom developer tools.

Antigravity CLI supports both local processes and remote host MCP server configurations.

Accessing the MCP manager

Type `/mcp` inside the prompt panel and press `Enter` to open the interactive MCP Manager Overlay. This panel allows you to:

- View live status rings for active, disconnected, or loading servers.
- Manually reload server configurations or check connection logs.

Global and workspace server configs

Unlike legacy setups, Antigravity separates MCP definitions into dedicated, sparse configurations:

- Global server setups: Configured in `~/.gemini/antigravity-cli/mcp_config.json`.
- Workspace local setups: Configured in your active project under `.agents/mcp_config.json`.

json

content_copy

```
{
  "mcpServers": {
    "sqlite-explorer": {
      "command": "node",
```

```
      "args": [
        "/usr/local/bin/sqlite-mcp-server.js"
      ],
      "env": {
        "SQLITE_DB_PATH": "/var/data/app.db"
      }
    }
  }
}
```

warning

Remote Connection Schema: When declaring remote SSE or websocket-based MCP connections, you must define the `serverUrl` field. Legacy fields like `url` or `httpUrl` are not supported.

Next steps

Learn how to migrate your existing configurations from Gemini CLI and troubleshoot connection anomalies:

- Migration from Gemini CLI: Fast-track your legacy extensions and config conversions.
- Troubleshooting: Resolve terminal hook errors, lockouts, or network failures.
- Permissions & Sandbox: Configure security containment rings around your custom plugins and MCP servers.

AI Credits

Status Line Customization

On this Page

Plugins & skills

The extensibility model

Antigravity plugins

Agent skills

Managing hooks

Model Context Protocol (MCP)

Next stProductskeyboard_arrow_down

Use Caseskeyboard_arrow_down

Pricing

Blog

Resourceskeyboard_arrow_down

Downloaddownload

Home

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

Overview

AI Credits

Customizations

Plugins & Skills

Status Line

Window Title

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Status line customization

Toggle standard status line components, define custom scripting configurations, and format dynamic JSON state payloads.

Overview

The status line is positioned at the bottom of the TUI prompt panel. It provides at-a-glance context regarding active agent cycles, workspace environments, context token window usages, and background execution tasks.

Interactive toggling

1. Type `/statusline` in the prompt box and press `Enter`.
2. The interactive Status Picker Panel overlay opens.
3. Use `↑`/`↓` to toggle specific metric elements (e.g., active model, task counters, context percentage) on and off.
4. Press `Enter` to commit your selection and exit. Press `Esc` to cancel.

Custom status line scripting

For advanced terminal layouts or custom status bar displays, you can route active agent metadata into a custom script.

Configuration

Add a `statusLine` configuration block to your `~/.gemini/antigravity-cli/settings.json` file:

json

content_copy

```
{
  "statusLine": {
    "type": "command",
    "command": "~/.gemini/antigravity-cli/statusline.sh"
  }
}
```

Whenever the agent state changes, the TUI executes your command script, pipes a detailed state JSON payload directly to the script's `stdin`, reads your formatted string from `stdout`, and renders the result in the prompt's status line. Full ANSI color codes are supported.

Available JSON fields

The JSON payload piped to your script contains the following top-level fields:

Field	Type	Description

`cwd`	string	Current working directory
`conversation_id`	string	Current conversation ID
`model`	object	`id` and `display_name` of the active model
`product`	str	Application name (e.g., `antigravity-cli`)

	i n g	
`workspace`	o b j e c t	`current_dir` and `project_dir` paths
`version`	s t r i n g	CLI version string
`plan_tier`	s t r i n g	Subscription tier of the authenticated user

`email`	string	LDAP/email of the authenticated user
`agent`	object	Active agent profile name
`context_window`	object	`total_input_tokens`, `total_output_tokens`, `context_window_size`, `used_percentage`, `remaining_percentage`, `current_usage`
`agent_state`	str	Current state: `idle`, `thinking`, `working`, `tool_use`, `initializing`

	string	
`vcs`	object	Version control info: `type` (git/jj/fig), `branch`, `client`, `dirty`
`sandbox`	object	Sandbox config: `enabled`, `allow_network`
`subagents`	array	Active subagent sessions with `name`, `role`, `status`

`artifacts`	array	Produced artifacts with `uri`, `status`, `type`
`pending_input_count`	int	Number of queued user messages
`background_tasks`	array	Running tasks with `name`, `status`, `index`
`tool_confirmation_pending`	bool	Whether a tool confirmation dialog is showing

`terminal_width`	int	Live width of the interactive terminal

JSON payload example

Here is a fully sanitized, typical JSON payload piped to your status line script:

json

content_copy


```
{
  "cwd": "/home/user/my-project",
  "conversation_id": "12345678-abcd-ef01-2345-6789abcdef01",
  "model": {
    "id": "Gemini",
    "display_name": "Gemini"
  },
```

```
"workspace": {
  "current_dir": "/home/user/my-project",
  "project_dir": "file:///home/user/my-project"
},
"version": "2026.04.15",
"context_window": {
  "total_input_tokens": 88244,
  "total_output_tokens": 61074,
  "context_window_size": 1048576,
  "used_percentage": 8.415603637695312,
  "remaining_percentage": 91.58439636230469,
  "current_usage": {
    "input_tokens": 63382,
    "output_tokens": 346,
    "cache_creation_input_tokens": 0,
    "cache_read_input_tokens": 20857
  }
},
"product": "antigravity-cli",
"agent_state": "idle",
"vcs": {
  "type": "git",
  "client": "my-project",
  "branch": "dev",
```

```
  "dirty": false
 },
 "sandbox": {
  "enabled": false
 },
 "plan_tier": "Pro",
 "email": "developer@email.com",
 "terminal_width": 111

}
```

Example script

You can download a complete, layout-adaptive script from the official statusline.sh example on GitHub. This script renders state badges, handles active branches, and formats context window progress bars dynamically.

Save the script to `~/.gemini/antigravity-cli/statusline.sh` and make it executable:

```
bash

content_copy
```

```
chmod +x ~/.gemini/antigravity-cli/statusline.sh
```

See also

- Terminal Title Customization: Configure dynamic window titles.
- Settings, Rendering & Keybindings: Customize keyboard hotkeys and buffers.
- Permissions & Sandbox: Manage secure directory permissions.

Status line customization

Overview

Interactive toggling

Custom status line scripting

See also

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

Overview

AI Credits

Customizations

Plugins & Skills

Status Line

Window Title

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

Terminal title customization

Configure dynamic window titles, map custom scripting configurations, and format JSON state outputs to customize terminal headers.

Overview

The terminal window title feature displays agent details, active workspace basenames, and active conversation parameters inside your terminal emulator's title bar. This lets you monitor agent progress even when the terminal window is minimized or unfocused.

Interactive toggling

- Syntax:

content_copy

```
/title [on|off]
```

- Type `/title` in the prompt box and press `Enter` to toggle the feature on and off.
- Type `/title on` or `/title off` to set the state explicitly.

Custom title scripting

For customized window title formatting, you can route active TUI state details into a custom shell script.

Configuration

Add a `title` configuration block to your `~/.gemini/antigravity-cli/settings.json` file:

json

content_copy

```
  {
    "title": {
      "type": "command",
      "command": "~/.gemini/antigravity-cli/title.sh"
    }
  }
}
```

Whenever the agent state changes, the TUI executes your command script, pipes a detailed state JSON payload directly to the script's `stdin`, reads your formatted string from `stdout`, and updates your terminal window title. Non-printable characters and ANSI escape sequences are automatically stripped before rendering.

JSON state payload schema

The JSON state payload is the same as the one sent to the custom status line script. It includes detailed properties representing `cwd`, `conversation_id`, `agent_state`, `vcs` details, and more. See the Status Line Schema for the complete property list.

Example script

You can download a complete, layout-adaptive script from the official title.sh example on GitHub. This script extracts the active workspace folder basename and renders a structured terminal title containing live agent states and conversation session prefixes.

Save the script to `~/.gemini/antigravity-cli/title.sh` and make it executable:

bash

content_copy

```
chmod +x ~/.gemini/antigravity-cli/title.sh
```

See also

- Status Line Customization: Customize dynamic TUI status bars.
- Settings, Rendering & Keybindings: Customize keyboard hotkeys and buffers.
- Permissions & Sandbox: Manage secure directory permissions.

Status Line Customization

Best Practices

On this Page

- Terminal title customization
- Overview
- Interactive toggling
- Custom title scripting
- See also

Resourceskeyboard_arrow_down

Downloaddownload

Home

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

Overview

AI Credits

Customizations

Plugins & Skills

Status Line

Window Title

Best Practices

Troubleshooting

Reference

Best practices for Antigravity CLI

Master the workflows, prompt architectures, and local configuration choices to maximize agent velocity while maintaining robust control.

Establish verification loops

The single most effective way to ensure reliable, correct modifications from an autonomous agent is to provide the agent with a local verification mechanism (such as unit tests, build commands, or formatting scripts).

Before asking the agent to implement a code change:

1. Ensure your workspace directory has a test suite ready.
2. If tests do not exist, direct the agent to write a standard test block *first*.
3. Once the agent proposes code, instruct it to run the local test command to verify its work.
4. Watch the agent execute the command and iterate on the test outputs automatically.

text

content_copy

> Implement feature X in main.py. Run npm test afterward to verify the build.

Explore, plan, then execute

Autonomous local agents operate with highest accuracy when complex changes are partitioned into distinct exploration, planning, and execution phases.

- Exploration: Ask the agent to explain how the target codebase resolves a particular problem or where an interface is defined before writing any changes.
- Planning: Request an implementation plan. The agent will list targeted files, required dependencies, and logic overrides in an implementation plan artifact.
- Execution: Once you approve the structured plan, direct the agent to apply the edits.

text

content_copy

> Explore how our router resolves `/docs/ page`. Write down an implementation plan to add `/docs/best-practices`.

Enrich your prompting context

Give local agents high-fidelity indicators to narrow down reasoning boundaries and minimize token overhead.

Target file autocompletion

Type `@` within your prompt box to trigger the Interactive Path Suggestion overlay. Highlighting and selecting a path imports the absolute workspace file path directly into your prompt. This helps the agent target its code searches.

Attaching visual evidence

If debugging visual UI issues, rendering bugs, or frontend layout inconsistencies, capture a screenshot or video recording, copy it, and press `ctrl+v` inside the prompt box to attach it. The agent will consult the media file to diagnose the issue.

Configure your workspace environment

Optimize your local workstation rules and security boundaries to match your engineering flow.

Write a codebase rule file

Create a `GEMINI.md` or `AGENTS.md` file at your workspace root to outline specific directory standards, styling paradigms, test command parameters, and deprecation warnings. The agent automatically parses these rules on startup and consults them before suggesting changes.

Establish structured permissions

Tune your safety barriers in `~/.gemini/antigravity-cli/settings.json` based on your project risk level:

- `request-review` (Default): Prompts you before executing any write operations, bash commands, or remote network calls.
- `proceed-in-sandbox`: Restricts all terminal executions to a secure sandbox containment ring. Safe commands execute autonomously, while risky commands prompt for reviews.
- `strict`: Always prompts for all non-read operations, providing complete line-by-line transparency.

```
json

content_copy

{

  "toolPermission": "proceed-in-sandbox",

  "enableTerminalSandbox": true

}
```

Manage TUI sessions proactively

Use active session navigation tools to recover from engineering dead-ends or course-correct intermediate agent loops.

Course-correct early (`esc`)

If you watch an agent execute an incorrect search pattern or write code that deviates from your intentions, press the global escape hatch key `esc` immediately to interrupt the turn and regain focus of a clean prompt.

Rewind history with `/rewind`

If an agent has made several successive changes that introduce build errors, you do not need to discard the session. Type `/rewind` (or `/undo`) to roll back your conversation thread to a previous stable checkout.

Branch experiments with `/fork`

If you are unsure of the best implementation path:

1. Reach a stable baseline thread.
2. Type `/fork` to spin up a duplicate parallel session.
3. Test your speculative code modifications in the branched session.
4. If the approach fails, run `/resume` to swap back to your stable main branch.

Automate and script

Antigravity CLI is designed to operate seamlessly within standard shell pipeline tools.

Run non-interactive commands (`-p`)

To automate quick queries or integrate agents into git hooks, use the one-shot prompt flag `-p`:

bash

content_copy

```
agy -p "Review this git diff and draft a conventional commit message" --cwd $(pwd)
```

Fan out using parallel subagents

For large-scale sweeps or multi-file refactoring, direct the primary agent to spawn concurrent background subagents. The agent manager handles background threads autonomously while you continue working on your primary screen.

Related resources

Learn how to configure settings and customize visual layouts:

- Settings, Rendering & Keybindings: Customize keyboard hotkeys and buffers.
- Permissions & Sandbox: Enforce filesystem containment.
- Plugins & Skills: Create your own custom slash commands.

Terminal Title Customization

Troubleshooting

On this Page

Best practices for Antigravity CLI

Establish verification loops

Explore, plan, then execute

Enrich your prompting context

Configure your workspace environment

Manage TUI sessions proactively

Automate and script

Related resources

Home

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

Overview

AI Credits

Customizations

Plugins & Skills

Status Line

Window Title

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity CLI
>Troubleshooting

Troubleshooting

Diagnose and resolve common anomalies with installation PATHs, local self-updating locks, keyring access permissions, and SSH clipboard forwarding.

Quick reference

Scan the lookup table below to identify symptoms and access immediate solutions:

Error Symptom	Potential Cause	Target Resolution
`agy: command not found`	Binary directory missing from shell environments.	Configure your shell PATH
`keyring: secure lock out`	Missing system service permissions or active lockouts.	Authorize keyring permissions
`SSH Clipboard paste failures`	Protocol streams blocked or missing forward configurations.	Enable emulator clipboard forwarding
`Advisory lock / update failures`	Locked self-updater thread or read-only directory paths.	Resolve self-updater locks and failures

---

Configure your shell PATH

Symptom

Executing `agy` returns a shell terminal error:

bash

content_copy

```
bash: agy: command not found
```

Cause

The installation utility downloads the binary to `~/.local/bin` (or `C:\Users\<Username>\AppData\Local\agy\bin`), but your shell's active `$PATH` environment does not index this directory.

Resolution

Ensure your terminal session loads the binary path.

macOS & Linux:

1. Open your shell configuration file (`~/.bashrc` or `~/.zshrc`).
2. Verify or append the following line at the end of the file:

content_copy

```
export PATH="~/.local/bin:$PATH"
```

1. Reload your profile configurations:

content_copy

```
source ~/.zshrc
```

Windows (PowerShell):

1. Open a PowerShell terminal as an Administrator and execute:

content_copy

```
 [System.Environment]::SetEnvironmentVariable("Path", [System.Environment]::GetEnvironmentVariable("Path", "User") + ";C:\Program Files\Google\antigravity-cli", "User")
```

1. Restart your terminal emulator for the system registry environment to refresh.

Authorize keyring permissions

Symptom

When launching, the CLI hangs, prints DBUS warnings, or throws keyring access exceptions:

text

content_copy

```
Error: failed to retrieve token: secret keyring is locked
```

Cause

Antigravity CLI utilizes secure keychain libraries (Apple Keychain, Linux secret-service via dbus, or Windows Credential Manager) to encrypt your session tokens. If the background daemon is locked or headless, the CLI cannot read credentials.

Resolution

macOS:

1. Open Keychain Access app.
2. Search for the `Antigravity CLI` security item.
3. Right-click, select Get Info, choose the Access Control tab, and verify that `agy` is on the allowed applications list.
4. If running inside a headless SSH session on Mac, run the following unlock sequence:

content_copy

```
security unlock-keychain -p "your_keychain_password" login.keychain
```

Linux:

Ensure your system keyring (such as GNOME Keyring or KWallet) is unlocked and accessible.

If you are running in a headless environment or over SSH, ensure that a D-Bus session is active and that your keyring daemon is running. You can typically initialize a D-Bus session by running:

```
bash

content_copy

export $(dbus-launch)
```

If you still experience access issues, ensure your user account has the necessary permissions to access the keyring service or reach out to support.

Enable emulator clipboard forwarding

Symptom

Pasting screenshots or media files via `Ctrl+V` within an SSH terminal returns a failure notification:

```
text

content_copy

Error: local pasteboard is empty or unreachable over SSH connection
```

Cause

Standard SSH streams do not forward graphical clipboards. Graphic uploads require specific terminal multiplexer protocols.

Resolution

Verify that you are utilizing supported terminal emulators and configurations.

1. Use iTerm2 or Ghostty: These emulators support advanced clip channels.
2. Configure iTerm2 Forwarding:

- Open iTerm2 Preferences (`Cmd+,`).

- Go to the General tab, select Selection submenu.
- Check Applications in terminal may access clipboard (enabling OSC 52 write channels).

1. Bypass Multiplexers: If running inside `tmux`, ensure your active configuration maps standard paste clips correctly:

content_copy

```
set -s set-clipboard on
```

Resolve self-updater locks and failures

Symptom

Launching `agy` hangs, fails to apply upgrades, or returns an advisory lock warning:

text

content_copy

Warning another background updater process is already active (update.lock)

Cause

Antigravity CLI contains a native, statically linked self-updater that runs in the background. It uses a 15-minute Time-To-Live (TTL) debounce marker (`last_check.timestamp`) and an advisory lock (`update.lock`) inside `~/.gemini/antigravity-cli/updater/` to prevent concurrent process collisions. If a background updater process hangs, crashes without releasing the lock, or has insufficient user filesystem permissions inside the executable directory, subsequent updates are blocked.

Resolution

- Release the advisory lock: Purge the background lock file manually:

content_copy

```
rm -f ~/.gemini/antigravity-cli/updater/update.lock
```

- Opt-out/Disable auto-updates: Set the `AGY_CLI_DISABLE_AUTO_UPDATE` environment variable to `true` inside your shell profile (`~/.bashrc` or `~/.zshrc`):

content_copy

```
export AGY_CLI_DISABLE_AUTO_UPDATE=true
```

- Verify directory write permissions: Ensure your user profile owns and has write permissions inside the target installation directory (`~/.local/bin/` on Unix, or `%LOCALAPPDATA%\agy\bin` on Windows).

Next steps

Access our quick reference sheets or configure advanced permissions:

- CLI Reference: Dense tables listing all slash commands and visual settings keys.
- Permissions: Configure fine-grained allowed and denied action policies.
- Sandbox: Enforce OS-level container isolation boundaries.
- Plugins & Skills: Create your own custom skills.

Best Practices

CLI Reference

On this Page

- Troubleshooting
- Quick reference
- Configure your shell PATH
- Authorize keyring permissions
- Enable emulator clipboard forwarding
- Resolve self-updater locks and failures
- Next steps

Productskeyboard_arrow_down

Use Caseskeyboard_arrow_down

Pricing

Blog

Resourceskeyboard_arrow_down

Downloaddownload

Home

Antigravity 2.0

Overview

Getting Started

Build with Google

Feature Overview

Models

Projects

Settings

Customizations

Agent Capabilities

Artifacts

Antigravity CLI

Overview

Getting Started

Installation & Auth

Tutorial

Using AGY CLI

Features

Gemini Migration

Prompting

Artifacts

Overview

Conversations

Agent Capabilities

Subagents

Permissions

Sandbox

Settings

Overview

AI Credits

Customizations

Plugins & Skills

Status Line

Window Title

Best Practices

Troubleshooting

Reference

Antigravity SDK

Antigravity IDE

Migration

Enterprise

Plans

FAQ

Antigravity CLI
>Reference

CLI reference

Scan scannable tables listing all TUI slash commands, default keyboard shortcuts, and JSON configuration parameters.

Core slash commands

Type `/` inside the prompt box to open the typeahead command selection menu.

Command	Category	Alias	Execution Purpose
`/add-dir <path>`	Utilities	—	Add a directory path to the active workspace.
`/agents`	Tools & Tasks	—	Open the Agent Manager Panel to monitor background subagents.

`/btw <query>`	Utilities	—	Ask a side question in the background without interrupting the main conversation.
`/clear`	Utilities	—	Clear the terminal and reset active conversation contexts.
`/config`	Configurations	`/settings`	Open the interactive Settings Editor Overlay.
`/diff`	Utilities	—	Show unified diff representations of all modified workspace files.
`/exit`	Core	—	Close the TUI session and restore your host shell.

`/fast`	Configurations	—	Enable fast mode (bypass reasoning plans) for quick actions.
`/fork`	Conversations	`/branch`	Clone the current conversation thread into a new parallel session.
`/hooks`	Tools & Tasks	—	Browse active pre-flight/post-format script hooks.
`/keybindings`	Configurations	—	Open the interactive Keyboard Shortcut Editor.
`/logout`	Account	—	Disconnect your profile and purge authentication tokens from the secure keyring.

`/mcp`	Tools & Tasks	—	Open the Model Context Protocol (MCP) server manager.
`/model`	Configurations	—	Choose your preferred reasoning model (persists across sessions).
`/open <path>`	Utilities	—	Force the path to open inside your default system editor.
`/permissions`	Configurations	—	Switch between global permission presets (`request-review`, `always-proceed`, `strict`).

`/planning`	Configurations	—	Enable multi-turn plan generation mode for complex engineering tasks.
`/rename <name>`	Conversations	—	Rename the current session thread.
`/resume`	Conversations	`/switch`, `/conversation`	Open the conversation picker overlay to select and load previous threads.
`/rewind`	Conversations	`/undo`	Roll back your conversation history to a previous message.
`/skills`	Tools & Tasks	—	Browse loaded local and global Agent Skills.

`/statusline`	Configurations	—	Open the Status Bar customization overlay.
`/tasks`	Tools & Tasks	—	Open the Task Manager Panel to monitor background shell execution logs.
`/title [on/off]`	Configurations	—	Toggle or set terminal window title updates.
`/usage`	Utilities	—	Launch the offline developer help manual inside the terminal.

Default keybindings

Keyboard shortcut commands mapping global, prompt, navigation, and approval operations.

Global controls

These hotkeys are always active regardless of which panel, overlay, or prompt is currently focused.

Key	TUI Command	Action Behavior
`Esc`	`cli.escape`	Closes active panels, halts active streams, or clears empty prompts.
`Ctrl+C`	`cli.exit`	Terminates the CLI session (prompts for confirmation if agent is working).
`Ctrl+L`	`cli.clear_screen`	Refreshes and clears the visual terminal buffer.

Prompt focus keys

These keys are active when writing instructions inside the prompt box.

Key	TUI Command	Action Behavior
`Enter`	`prompt.submit`	Submits your prompt or active menu selection to the agent.
`Shift+Enter` / `Ctrl+J`	`prompt.newline`	Inserts a clean newline without submitting.
`Ctrl+V`	`prompt.paste`	Pastes graphic media files or clipboard blocks into the prompt.
`Ctrl+O`	`prompt.toggle_trajectory`	Expands or collapses detailed tool reasoning outputs.

`Ctrl+R`	`prompt.open_review`	Opens the Artifact Review Panel.
`Ctrl+G`	`prompt.external_editor`	Launches your default `$EDITOR` shell to compose your prompt.
`Alt+J`	`prompt.teleport_agent`	Instantly switches focus to the next subagent awaiting confirmation.
`Ctrl+K`	`prompt.fast_approve`	Instantly approves the pending subagent action listed in the status alert.
`Ctrl+A`	`prompt.cursor_start`	Moves the prompt insertion cursor to the beginning of the line.

`Ctrl+E`	`prompt.cursor_end`	Moves the prompt insertion cursor to the end of the line.
`Ctrl+Z`	`prompt.undo_text`	Reverts the last edit.
`Ctrl+Shift+Z`	`prompt.redo_text`	Redoes the last undone text operation.

Navigation & scrolling

Used inside select panels, menus, and scrollable text boxes.

Key	TUI Command	Action Behavior

`↑` / `↓`	`navigation.up` / `navigation.down`	Scrolls highlighted selections up or down by one item.
`PgUp` / `Shift+↑`	`navigation.page_up`	Scrolls the active text viewport up by one page block.
`PgDn` / `Shift+↓`	`navigation.page_down`	Scrolls the active text viewport down by one page block.
`←` / `→`	`navigation.left` / `navigation.right`	Swaps pages inside multipage structures (like the Session Picker).
`Tab`	`navigation.tab`	Confirms the highlighted slash-command autofill option.

Tool confirmations

Active during confirmation prompts.

Key	TUI Command	Action Behavior
`y`	`confirm.yes`	Authorizes the proposed tool, command, or active artifact.
`n`	`confirm.no`	Rejects the proposed tool, command, or active artifact.
`A`	—	(Inside Review Panel) Approves all generated artifacts in one action (built-in shortcut).

Configuration keys (`settings.json`)

Primary settings key names, data types, system defaults, and expected parameters.

Option Key Name	Value Type	System Default	Parameter Characteristics & Options
`colorScheme`	string	`"terminal"`	Color theme: `"light"`, `"solarized light"`, `"colorblind-friendly light"`, `"dark"`, `"solarized dark"`, `"colorblind-friendly dark"`, `"tokyo night"`, or `"terminal"` (inherits native shell colors).
`altScreenMode`	string	`"default"`	Screen buffer usage: `"default"` (native terminal inline), `"always"` (altscreen no-flickering buffer).

`toolPermission`	string	`"request-review"`	Global safety presets: `"request-review"` (prompts for write/bash/web tools), `"proceed-in-sandbox"` (auto-proceed inside sandbox), `"always-proceed"` (never prompts), or `"strict"` (prompts for all non-read tools).
`artifactReviewPolicy`	string	`"asks-for-review"`	Code review policy: `"asks-for-review"` (always prompts before writing code), `"agent-decides"` (prompts dynamically), or `"always-proceed"` (never prompts).
`notifications`	boolean	`false`	Emits system desktop and terminal bell chime notifications upon task completions.

`showTips`	boolean	`true`	Displays helpful agentic tips above the prompt panel during generation turns.
`showFeedbackSurvey`	boolean	`true`	Displays periodic quality feedback surveys upon active task completions.
`editor`	string	`"auto"`	Target text editor utility: `"auto"` (consults system `$EDITOR`), `"vim"`, `"emacs"`, or custom text labels.
`allowNonWorkspaceAccess`	boolean	`false`	Permits the agent's file read and write tools to navigate outside recognized Git/workspace roots.
`enableTerminalSandbox`	boolean	`false`	Restricts all local execution commands launched by agents to OS containment rings.

`enableTelemetry`	boolean	`true`	Permits metric collection and crash log streaming to improve tool reliability.
`verbosity`	string	`"high"`	Visual verbosity level: `"high"` (renders full thoughts and tool outputs) or `"low"` (displays only minimal visual progress indicators).
`runningLightSpeed`	string	`"medium"`	Visual running light progress animation speed: `"fast"`, `"medium"`, `"slow"`, or `"off"`.

Next steps

Learn how to safely deploy permission policies, sandboxes, and customize plugins:

- Permissions & Sandbox: Enforce command-line containment rules.
- Plugins & Skills: Create your own custom skills slash commands.

- Installation & Auth: Update your CLI install.

Troubleshooting

Overview + Quick Start

On this Page

CLI reference

Core slash commands

Default keybindings

Configuration keys (`settings.json`)

Next steps